AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 5

MCP: Integration, Servers and Errors

Every failure said "Operation failed." So the agent retried a refund that policy forbade, three times, then told the customer to try again later. A service outage and a permanent business rule looked identical from where it stood.

Module	AI06 of 13
Lesson	5 of 10
Prerequisites	AI06 L3–L4
Exam relevance	**Direct.** All of TS 2.2 and TS 2.4, and TS 5.3's error propagation
Reading time	~40 minutes

Contents

1. MCP: Integration, Servers and Errors

Lesson 5 of 10 · Agents and MCP

MCP: Integration, Servers and Errors

Key idea

An error has to tell the agent what to do next, or the agent can only guess. Separate transient, validation, business and permission failures, and say whether a retry can help. **An empty result isn't a failure, and a failure must never look empty.** Configure team servers in `.mcp.json`, with secrets expanded from the environment, and reuse community servers for standard systems.

> *Before: MCP primitives (AI06 L4). Teaches: error categories, local recovery, propagation with context, config scopes, env expansion, custom vs community servers, and the API's MCP connector. Used by: AI06 L9–L10, AI08 L4, AI12.*

The hook: what a generic error costs

Here are six scripted tools, handled two ways. The first treats all failures alike. The second reads a category and a retry flag. Real output from `code/ai06-agents/.../ToolErrors.java`:

```
Scenario             generic   typed    typed action
timeout, then fine   3         3        use result
service stays down   3         3        report to coordinator, with what was tried
bad date format      3         1        correct the input
refund over policy   3         1        explain to the customer
not authorized       3         1        escalate
no matching orders   1         1        use result
Total tool calls     16        10
```

> *Key idea*
>
> **The typed version didn't just save 6 calls. It also knew what to do next.** The generic version ends every failing path with "give up", so a customer asking about an over-limit refund gets the same answer as during an outage. These counts come from one script, so the point is the behavior, not the numbers.

Intuition: the hospital receptionist

"The doctor can't see you" means very different things:

What's really going on	What you should do
The doctor's in another room, for ten minutes	Wait, then ask again
You filled in the wrong form	Fix the form
This clinic doesn't treat that condition	Stop asking, and go elsewhere
You aren't registered here	Get authorization

A receptionist who only ever says "Not possible" sends everyone back to the waiting room.

Mechanism: two kinds of MCP error

The MCP spec separates these:

Kind	For	How it's reported	Can the model fix it?
Protocol error	Unknown tool, malformed request, server error	JSON-RPC `error`	Less likely
Tool execution error	API failures, invalid input, business rules	`result` with `isError: true`	Yes, so it **should** be given to the model

Categories for tool errors

Category	Examples	Retryable?	The agent should
Transient	Timeout, 503, rate limit	**Yes**	Retry with backoff, then report
Validation	Wrong date format, missing field	Only with corrected input	Fix its input
Business	Refund over limit, policy violation	**No**	Explain, or take the alternative path
Permission	Not allowed, expired credentials	**No**	Escalate, or report

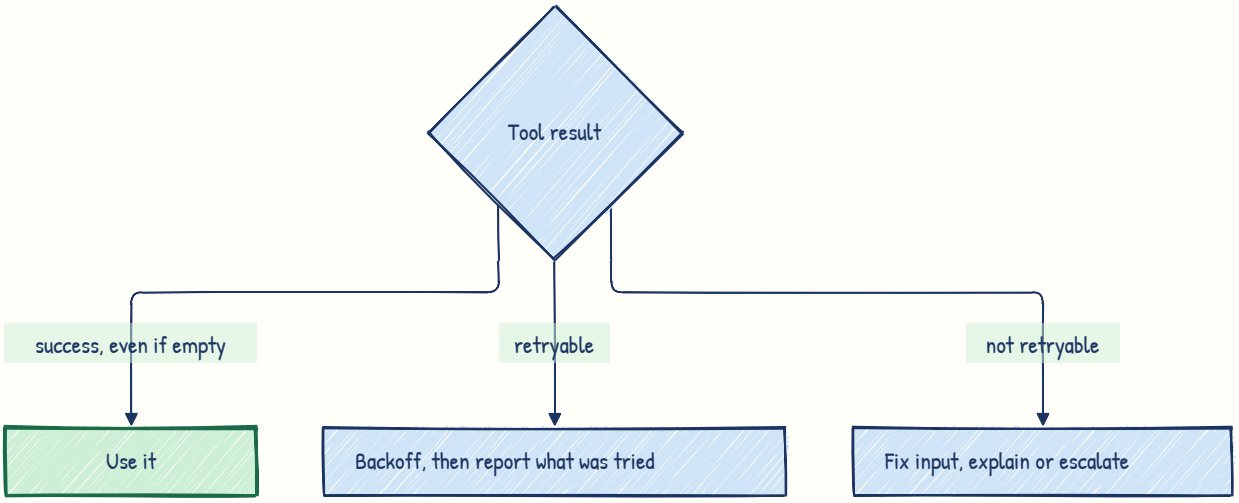


Figure 1. Deciding what to do with a tool result. Only transient errors justify retrying the same call, and an empty result is a success.

> **`errorCategory` is a convention, not a protocol field**
>
> `isError` is in the MCP spec. `errorCategory` and `isRetryable` come from the exam guide, as metadata you design. They travel inside the result's content. Don't expect an SDK field for them. You could also use `structuredContent` with an `outputSchema`.

Real output of the over-limit case, as it would sit in `content`:

```
{"errorCategory":"business","isRetryable":false,"message":"Refunds over $500 need a manager"}
```

The message itself is for the customer-facing agent to use, so write it to be repeatable to a person, with no internal detail.

Empty is not failed

This is one of the most common exam traps.

Case	Result	What the agent can conclude
Search ran, and found nothing	Success, 0 results	"No such orders exist"
Search service timed out	Error, transient	"I don't know yet"
Timeout **returned as** empty	Success, 0 results	**"No such orders exist", which is false**

The third row is the disaster. The agent confidently tells a customer that their order doesn't exist, and nothing in your logs looks wrong.

Recover locally, propagate the rest

In a multi-agent system, **TS 2.2 and TS 5.3** say a subagent handles what it can, and passes up only what it can't.

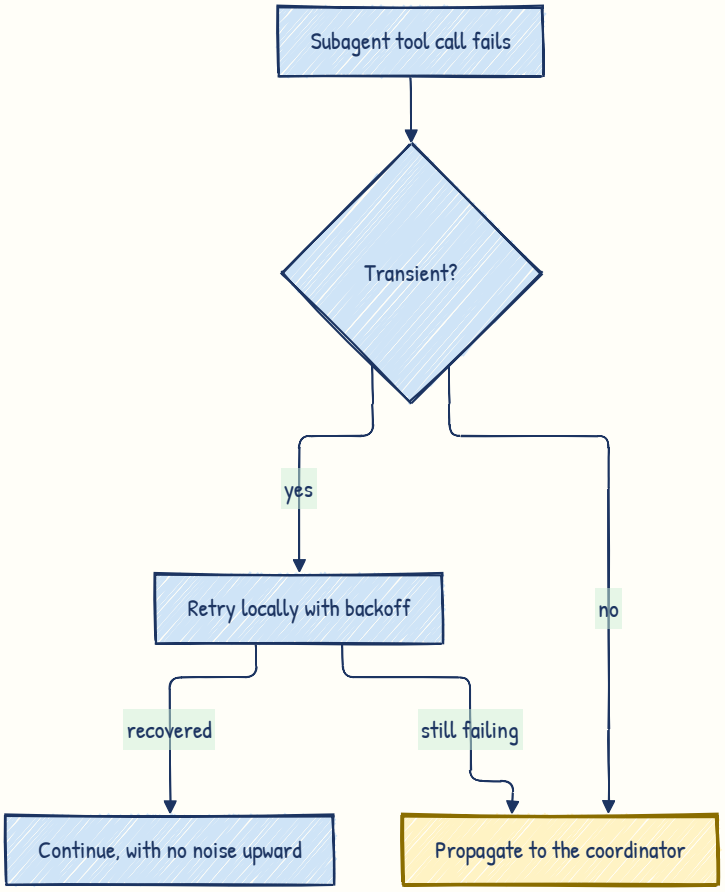


Figure 2. Error handling across agents. The subagent retries transient failures itself. The coordinator hears only what's unresolved, with enough context to decide.

A useful propagated error has four parts:

Field	Example
Failure type	Transient, payer portal
What was attempted	Query "coding audits 2025", tried twice
Partial results	1 finding from a journal search
Alternatives	Try a different source, or proceed and note the gap

Wrong answer	Why it's wrong
Return empty as success	Hides the failure
Kill the whole workflow	Discards everything else's work
Generic "search unavailable"	Removes the coordinator's options
Retry forever	Wastes time and budget

Configuring MCP in Claude Code

From the Claude Code MCP docs:

Scope	Stored in	Shared?	Use for
Local (default)	`~/.claude.json`, under the project path	No	Your experiments in one project
Project	`.mcp.json` in the repo root	**Yes, through version control**	Team servers
User	`~/.claude.json`	No	Your personal servers, across projects

If a name is defined in several places, Local beats Project, which beats User. Whole entries win, and fields aren't merged.

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "${GITHUB_MCP_URL:-https://mcp.example.com}/mcp",
      "headers": { "Authorization": "Bearer ${GITHUB_TOKEN}" }
    },
    "orders-db": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@acme/orders-mcp"],
      "env": { "DB_URL": "${ORDERS_DB_URL}" }
    }
  }
}
```

Fact	Detail
`${VAR}` and `${VAR:-default}`	Work in `command`, `args`, `env`, `url` and `headers`
Missing variable, no default	The config still loads, with a warning, and the literal `${VAR}` text is used
Credential-named variables, such as `ANTHROPIC_API_KEY`	Read as empty in remote `url` and `headers`, so a project's `.mcp.json` can't exfiltrate them
Project servers	Need interactive approval. `claude -p` and the Agent SDK load them **without** prompting
Transports	HTTP recommended, stdio for local, SSE deprecated
Tool names	`mcp__<server>__<tool>`, used in permissions and hook matchers
Output	Warning above 10,000 tokens, default limit 25,000, changeable with `MAX_MCP_OUTPUT_TOKENS`

The `-p` row deserves a second look. In CI, a cloned repo's `.mcp.json` servers load with no prompt. Block them with `--strict-mcp-config`, or `disabledMcpjsonServers`, if you run untrusted repositories.

Should you build a server?

Situation	Choice	Why
Jira, GitHub, Slack, Postgres	**Existing community or vendor server**	Don't rebuild a standard integration
Your own refund workflow	**Custom server**	Nobody else has your rules
The agent keeps using Grep instead of your better MCP search	**Improve the MCP tool's description**	TS 2.4: explain capabilities and outputs in detail

The MCP connector on the Claude API

Outside Claude Code, the Messages API can call remote servers directly, with Anthropic making the connection. It's in beta (`mcp-client-2025-11-20`), and it needs **both** halves, or it fails validation. This shape comes from the bundled API reference, and hasn't been run:

```
{
  "mcp_servers": [{ "type": "url", "name": "orders", "url": "https://mcp.acme.example/mcp" }],
  "tools": [{ "type": "mcp_toolset", "mcp_server_name": "orders" }]
}
```

Why the obvious fixes fall short

Tempting move	Why it falls short	Better
Return "Operation failed"	The agent can't choose a recovery	Category, retry flag, message
Catch exceptions, and return `[]`	Failures look like empty results	`isError: true`
Retry everything three times	Wastes calls on permanent errors	Check the retry flag
Commit tokens to `.mcp.json`	Secrets end up in git	`${VAR}` expansion
Put team servers in `~/.claude.json`	Teammates don't get them	`.mcp.json`
Build your own Jira server	You now maintain Jira's whole surface	Use the existing one

How Anthropic and the field solved it

Problem	Approach	Source
Agents need to know a tool failed	Letting the agent know a tool is failing works well, and the research system combines this with retries and checkpoints	Anthropic's multi-agent post
Errors should be useful	Replace opaque codes with specific guidance and correct examples	Writing Tools for Agents
Tools fail in many ways	`isError` lets the model self-correct	MCP spec

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: exceptions become strings. The agent sees a stack trace, or nothing.

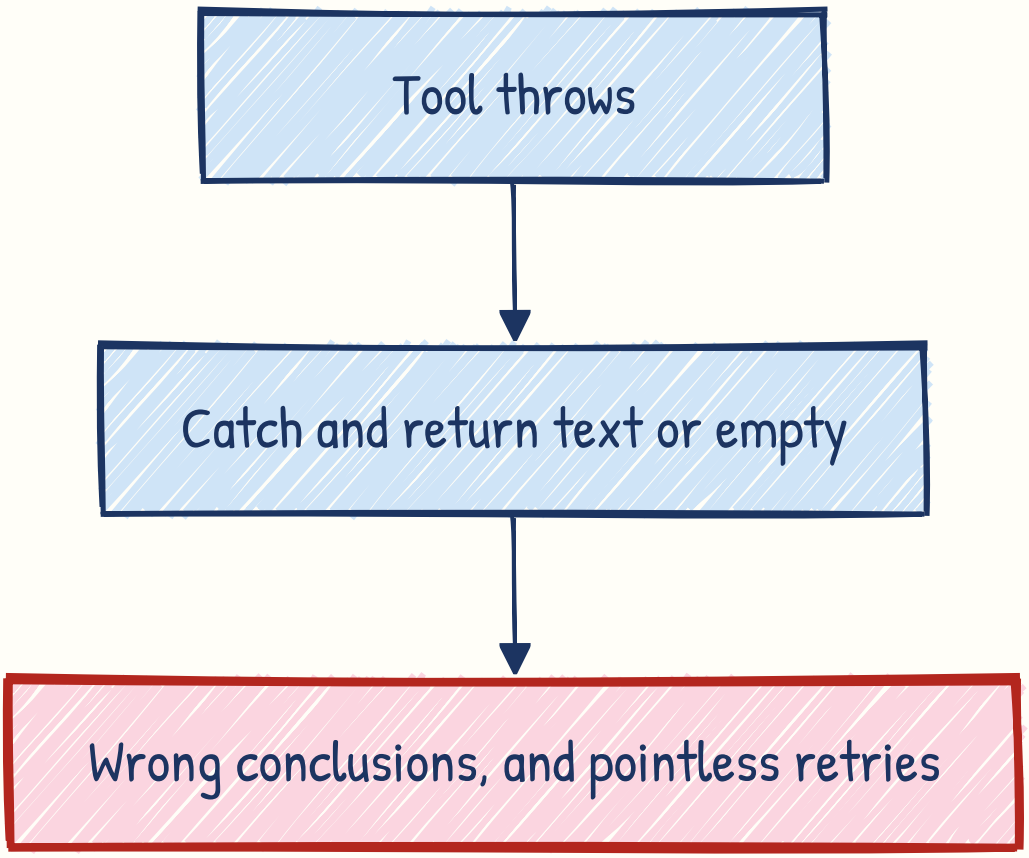


Figure 3. Baseline. Raw exceptions reach the model, or get swallowed, so it can't tell an outage from a rule.

Improved: structured errors. The agent recovers sensibly. New problem: each tool team invents its own categories, and a failing service still receives retries from many agents at once.

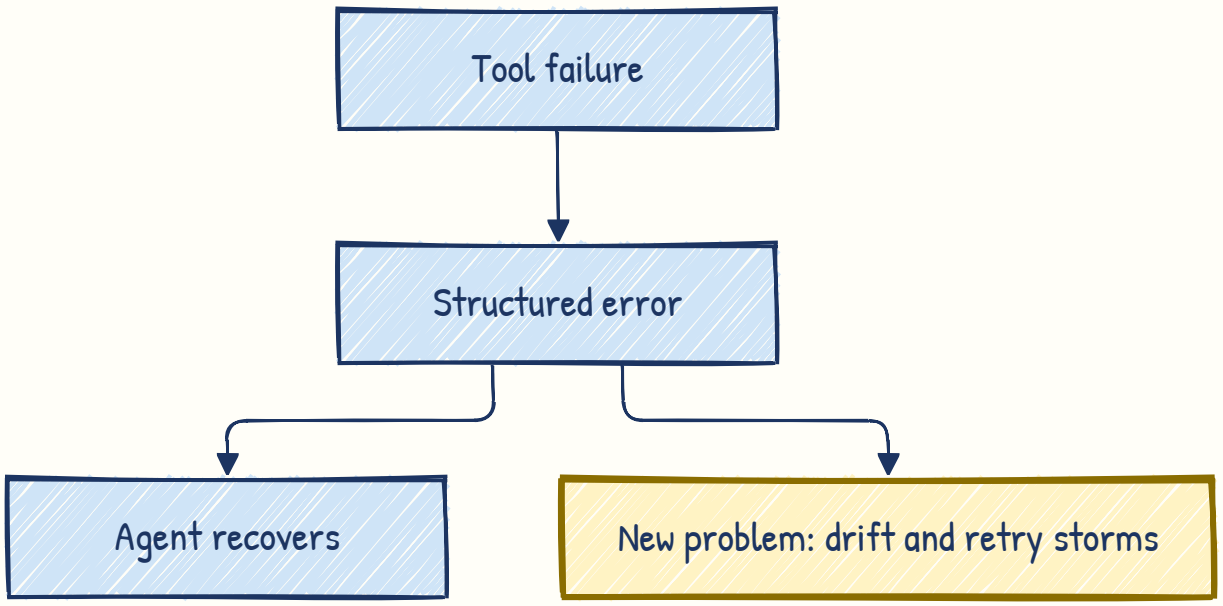


Figure 4. Improved. Structured errors help each agent, but categories drift, and retries can pile onto a struggling service.

Production: a shared error contract.

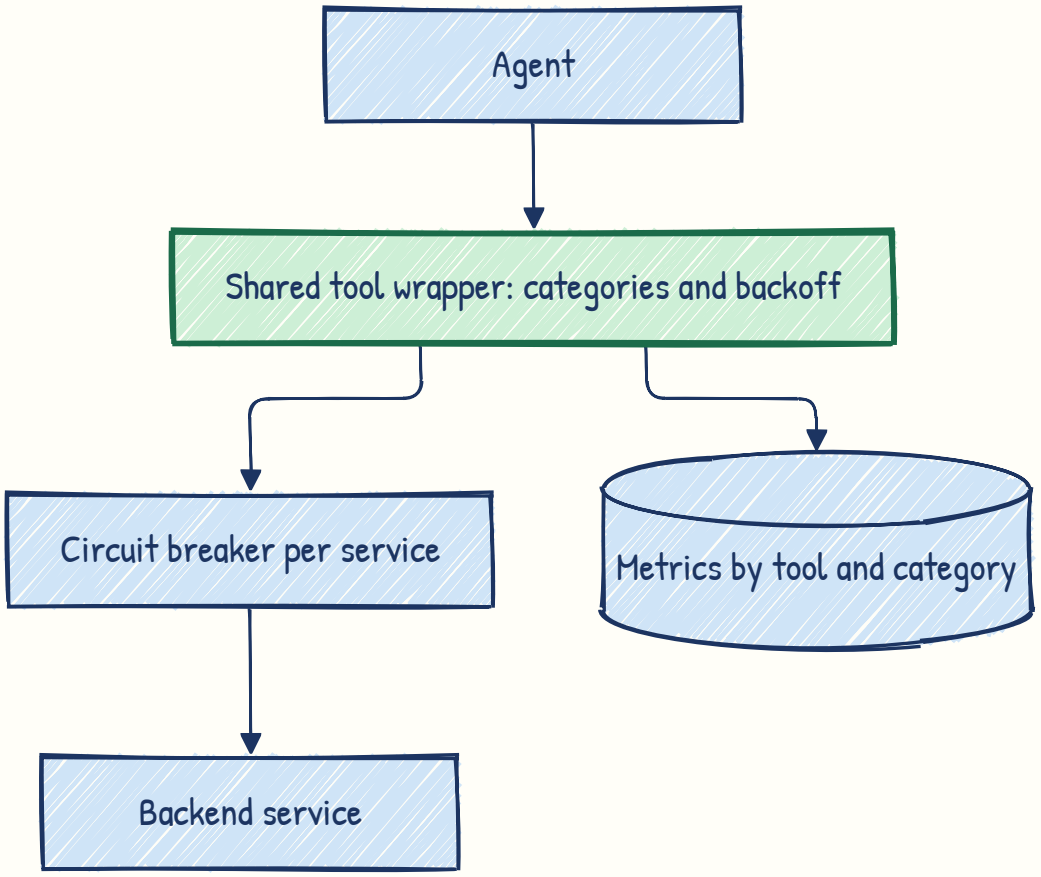


Figure 5. Production. A shared library enforces one error format. Circuit breakers stop retries against a failing service, and error rates by category are monitored.

Component	Why it exists	How it communicates
Tool wrapper	One format, and bounded retries	Library used by all servers
Circuit breaker	Protects a struggling service (HLD M12)	Fails fast with a transient error
Metrics	A rise in `validation` often means a bad description	Async
Idempotency keys	Retries can't repeat side effects	Passed to the backend

Failures

Failure	Containment
Retry storm	Circuit breaker, backoff with jitter
Secret missing from the environment	Fail clearly at startup, and alert
Community server upgrade changes a tool	Pin versions, and run evals on upgrade
Output too large	Paging, or truncate with guidance
Untrusted repo's `.mcp.json` in CI	`--strict-mcp-config`

Java

From `ToolErrors.java`: the retry decision reads metadata, and never message text.

```
result = tool.apply(calls);
calls++;
if (!result.isError()) {
    return new Report("use result", calls, result.message());
}
if (!result.retryable()) {
    break;
}
```

A real system adds backoff between attempts. The next action depends on the category:

```
return switch (result.category()) {
    case TRANSIENT -> "report to coordinator, with what was tried";
    case VALIDATION -> "correct the input";
    case BUSINESS -> "explain to the customer";
    case PERMISSION -> "escalate";
};
```

Trade-offs

Decision	Gain	Cost
Rich error metadata	Better recovery	Tokens, and discipline across teams
Local recovery	Less noise for the coordinator	Failures can be hidden unless logged
Community server	No upkeep of integration code	Supply chain trust, less control
Project scope	Everyone gets the same tools	Reviews needed, and auto-loads in `-p`

When simple errors are enough: a small tool with one failure mode can return a clear sentence and `isError`.

Common mistakes

1. **"Operation failed".** It offers nothing to act on.
2. **Returning empty on failure.** It creates false facts.
3. **Retrying business errors.** They'll fail every time.
4. **Marking no-results as an error.** It triggers pointless retries.
5. **Killing the workflow over one failure.** Pass partial results up.
6. **Committing tokens.** Use `${VAR}`.
7. **Expecting `isRetryable` from the protocol.** You design it.
8. **Forgetting `-p` loads project servers.** Restrict them for untrusted repos.

What to say in an interview

"Tool errors need a category and a retry flag. Transient errors get bounded retries in the subagent, validation errors get corrected, and business errors get explained. A timeout must never look like empty results, because the agent would state something false. Unresolved errors go to the coordinator with what was tried, partial results and alternatives."

Certification lens

How this lesson maps to CCAR-F

TS 2.2, TS 2.4 and TS 5.3. Structured errors with `errorCategory` and `isRetryable`; access failure vs valid empty; `.mcp.json` for the team and `~/.claude.json` for personal, with `${VAR}` secrets; community servers for standard systems.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Medium. `process_refund` returns "Operation failed" for timeouts and for policy violations alike. The agent retries both. What's the best fix?

A. Tell the agent to retry only once.

B. Have the agent analyze the customer's tone before retrying.

C. Return `isError` with a category, a retry flag and a clear message, so it can retry timeouts and explain policy limits.

D. Return success with a note in the text.

A C (TS 2.2). A treats both the same. B measures the wrong thing. D hides failure.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Medium. A search subagent's source times out repeatedly. What should it send the coordinator?

A. The failure type, the query attempted, partial results and possible alternatives.

B. An empty result marked as success, so that the run continues.

C. An exception that terminates the whole research run.

D. "Search unavailable."

A A (TS 5.3). B hides the failure, and leads to false conclusions. C discards other work. D removes the coordinator's options.

Q Q3 · Exam style, scenario: Code Generation with Claude Code · Easy. Your team needs a shared GitHub MCP server, and each developer has their own token. What's the right configuration?

A. Each developer adds it to `~/.claude.json` with the token inline.

B. Commit a shared token in `.mcp.json`.

C. Store the token in `CLAUDE.md`.

D. Add it to the repo's `.mcp.json` using `${GITHUB_TOKEN}`, with each developer setting that variable.

A **D** (TS 2.4). A isn't shared. B leaks a secret. C also leaks one, and isn't configuration.

Q Q4 · Exam style, scenario: Developer Productivity with the Agent SDK · Medium. The team needs a Jira integration. What's the best first choice?

A. Write a custom MCP server for full control.

B. Use an existing community or vendor server, keeping custom servers for team-specific workflows.

C. Call Jira's REST API with the Bash tool.

D. Export issues to files nightly.

A **B** (TS 2.4). A means maintaining a standard integration. C gives the harness only an opaque string to control. D goes stale.

Q Q5 · Debugging · Hard. An agent tells a customer "you have no orders." Logs show `lookup_orders` returned `[]`. The customer has 12 orders. What's likely wrong in the tool, and how would you prove it?

A The tool probably catches a backend error or timeout and returns an empty list. Check backend logs for errors at that time, and read the tool's exception handling. Fix: return `isError: true` with a transient category, and test by injecting a timeout. Add a metric for empty results, since a spike can reveal this.

Revision - Lesson 5

→ Tool execution errors use isError. Protocol errors use JSON-RPC.

→ Categories: transient, validation, business, permission.

→ errorCategory and isRetryable are your convention, not protocol fields.

→ Only transient errors justify retrying the same call.

→ Empty results are success. Failure must never look empty.

→ Subagents retry locally, then propagate type, attempt, partial results, alternatives.

→ Don't kill the workflow, and don't hide the failure.

→ .mcp.json: team, in version control. ~/.claude.json: personal.

→ Use dollar-brace variable expansion for secrets, and never commit tokens.

→ Local beats project, which beats user.

→ Prefer existing servers for standard systems, and custom ones for your workflows.

→ API connector needs mcp_servers and an mcp_toolset, plus the beta.

Your notes